HHZ Sommersemester 2026

Java Bootcamp

Kristof Werling, 202604

Von Stack zu DSL in einem Tag

VIBE CODING IN JAVA
MIT KI ZUR EIGENEN PROGRAMMIERSPRACHE

Was wir heute erreichen

- Wir bauen eine eigene Sprache
- Wir nutzen KI massiv zur Unterstützung
- Wir lernen, Kontrolle zu behalten
- Wir verstehen, wie Software wirklich entsteht

Was ist Vibe Coding?

- KI schreibt (fast) den gesamten Code
- Entwickler steuert über Prompts
- Fokus: Geschwindigkeit statt Perfektion

Das Risiko

- KI verändert bestehenden Code
- Funktionalität geht verloren
- Verständnis sinkt
- Kontrolle verschwindet

Unser Ansatz heute

- Kleine, kontrollierte Schritte
- Bestehenden Code schützen
- KI wie einen Junior behandeln
- Immer testen

Unser Projekt

- Eine stackbasierte Sprache (Forth-ähnlich)
- Sehr einfache Struktur
- Schrittweise Erweiterung
- Keine festen Datentypen
- Operationen definieren Verhalten

Was ist ein Stack?

- Eine einfache Datenstruktur
- Elemente werden „oben“ abgelegt
- Zugriff nur auf das oberste Element

Last In – First Out (LIFO)

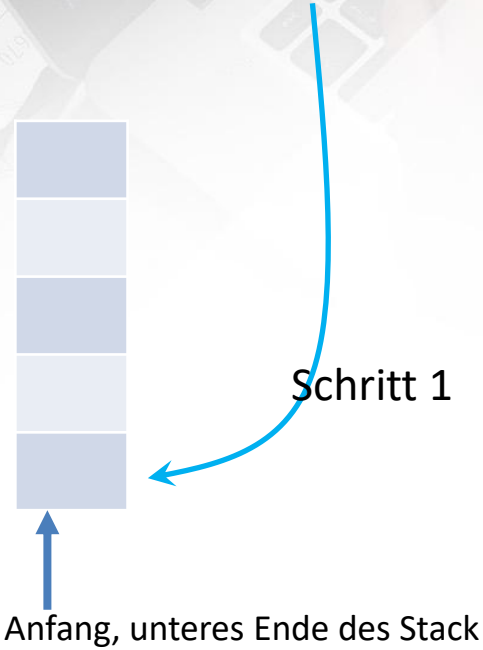
Beispiel mit Integern

Eingabe: 3 4 5

Anfang, unteres Ende des Stack

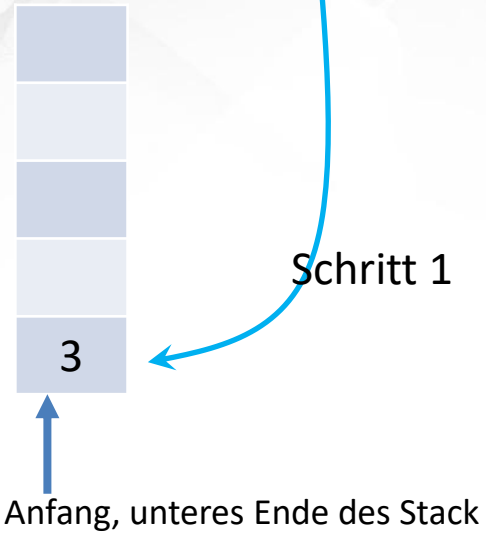
Beispiel mit Integern

Eingabe: 3 4 5



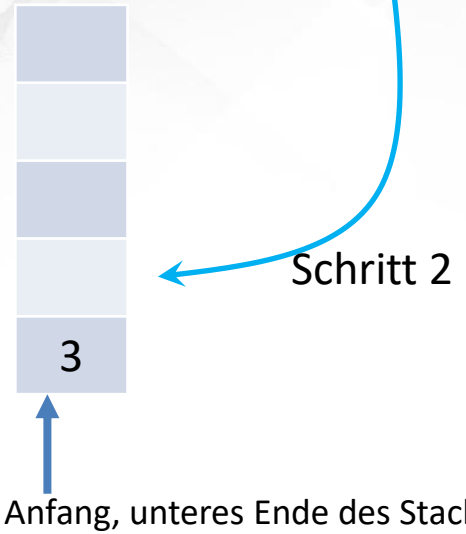
Beispiel mit Integern

Eingabe: 3 4 5



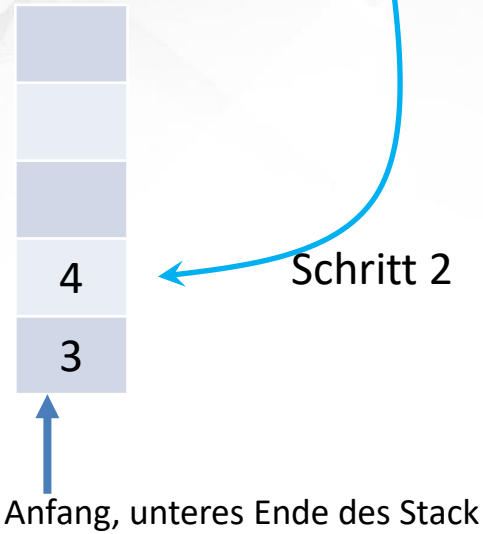
Beispiel mit Integern

Eingabe: 3 4 5



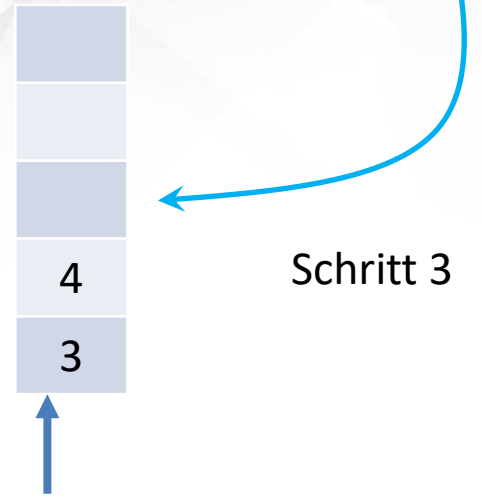
Beispiel mit Integern

Eingabe: 3 4 5



Beispiel mit Integern

Eingabe: 3 4 5

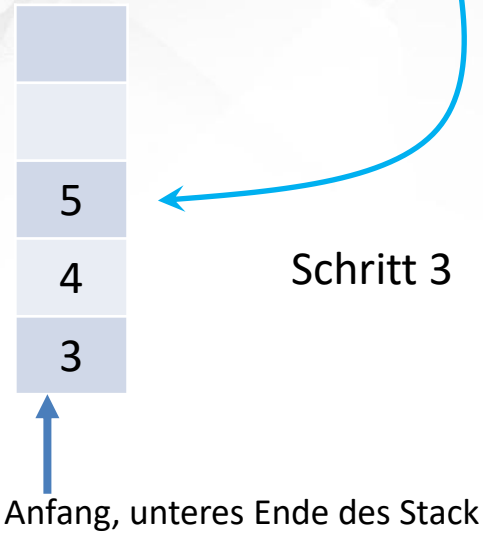


Schritt 3

Anfang, unteres Ende des Stack

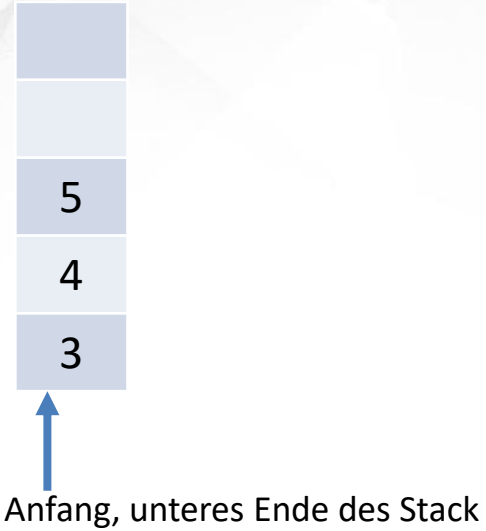
Beispiel mit Integern

Eingabe: 3 4 5



Beispiel mit Integern

Eingabe: 3 4 5



Diese Einträge können nur in umgekehrter Reihenfolge wieder vom Stack geholt werden.

Also zuerst die 5, dann die 4, dann die 3

Stack-Operationen

Grundoperation	Erklärung
push	Wert oben auf den Stack legen
pop	Obersten Wert vom Stack holen
peek	Obersten Wert auf dem Stack anschauen

Eingabe: 3 4 5

Stackoperationen:

push(3)
push(4)
push(5)

Auslesen:

pop() -> 5
pop() -> 4
pop() -> 3

Gute Nachricht

- Wenn die KI einen guten Job macht, dann müssen wir uns mit den Stack-Grundoperationen nicht auseinandersetzen.
- Wir betrachten den Interpreter aus Sicht der Programmiersprache selbst.

Jorth

- Jorth ist stark an die Programmiersprache Forth angelehnt (ebenfalls stack-basiert)
- Jorth: J für Java und orth für Forth
- Jorth setzt sich aus den Datenelementen-Elementen (die auf dem Stack sind) und Commands zusammen.
- Commands arbeiten mit den Daten-Elementen auf dem Stack

Jorth Beispiel

Eingabe: 3 4 +

3	Integer 3
4	Integer 4
+	Command Plus (= Addition)

Bei dieser Schreibweise spricht man auch von der Reverse-Polish-Notation oder
Der Umgekehrt-Polnischen-Notation.

Erste 2 Jorth Commands

Damit wir sinnvoll mit Jorth arbeiten können, brauchen wir diese beiden Commands

Command	Erklärung
+	Holt die beiden oberen Elemente vom Stack, addiert sie und speichert die Summe wieder zurück auf den Stack. = push(pop() + pop())
.	Holt das oberste Element vom Stack und druckt es auf die Console.

PRAXISTEIL

Vorgehen

- Sie bekommen einen ersten Prompt
- Wir führen ihn gemeinsam aus
- Wir analysieren das Ergebnis
- Danach arbeiten Sie selbstständig weiter and den weiteren Aufgaben.

GIT Repositories

- Meine Musterlösung wird in einem Git-Repository zur Verfügung gestellt.
- Ihre eigene Entwicklungsschritte speichern Sie bitte **regelmäßig / häufig** in einem eigenen Git-Repository.
- Falls notwendig kopieren Sie sich einzelne Dateien aus meinem Musterlösungs-Repository.

Verzeichnisstruktur pro Schritt

Struktur :

```
jorth-workshop/
├─ src/
│   └─ jorth/
│        ├─ Interpreter.java
│        ├─ StackMachine.java
│        └─ Main.java
├─ tests/
│   └─ jorth/
│        ├─ Step01Tests.java
│        ├─ Step02Tests.java
│        └─ TestSupport.java
├─ slides/
├─ README.md
├─ run-tests.bat
└─ run-tests.sh
```

Ausgangsbasis (Jorth.java)

```
public class Jorth {
    public static String run(String input) {
        if (input == null || input.isBlank()) {
            return "[]";
        }
        return "[]";
    }
    public static void main(String[] args) {
        String program = String.join(" ", args);
        System.out.println(run(program));
    }
}
```

Ausgangsbasis (JorthTest.java)

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;
public class JorthTest {
    @Test
    void emptyProgramReturnsEmptyStack() {
        assertEquals("[]", Jorth.run(""));
    }
    @Test
    void blankProgramReturnsEmptyStack() {
        assertEquals("[]", Jorth.run("   "));
    }
}
```

Kompilieren & Tests laufen lassen

- Windows: run-tests.bat
- Mac / Linux: run-tests.sh

Aufgabe

- Neuen Chat in einer GenAI ihrer Wahl starten
- Die beiden Java Files hochladen in den Chat

Nächster Schritt

- Jorth soll Integer einlesen und diese auf dem Stack ablegen können
- Dafür wollen wir den notwendigen Code erzeugen lassen
- Auch die neuen Tests sollen erzeugt werden

Wie könnte der Prompt aussehen?

- Was soll rein?
- Was nicht?
- Was ist unbedingt notwendig?
- Was könnte man reinnehmen, muss man aber nicht?

Beispiel-1

Schreibe die Methode run so, dass Zahlen aus dem Input auf den Stack gelegt werden.

Beispiel-2

Wir bauen eine einfache stackbasierte Sprache in Java.

Die Methode run(String input) soll Zahlen erkennen und auf einen Stack legen.

Tokens sind durch Leerzeichen getrennt.

Bitte implementiere diese Funktionalität.

Beispiel-3

Wir bauen eine einfache stackbasierte Sprache in Java namens Jorth.

Es gibt eine Methode run(String input), die aktuell noch nichts macht.

Aufgabe:Ergänze die Methode so, dass Ganzzahlen erkannt und auf einen Stack gelegt werden.Tokens sind durch Leerzeichen getrennt.

Beispiele:"3" → "[3]""3 4" → "[3, 4]"

Bitte schreibe einfachen und verständlichen Code.

Vergleich

Beispiel	Was fehlt	Verbesserung	Was macht die KI
1	• kein Kontext • kein aktueller Stand • keine Beispiele • keine Einschränkungen	-	• neue Klassen • komplexe Struktur • evtl. Parser • unnötige Logik
2	• keine Beispiele • keine Einschränkungen • keine Kontrolle über Änderungen	• Kontext vorhanden • Aufgabe klarer • weniger Interpretationsspielraum	KI kann immer noch „zu viel“ machen
3	• keine Einschränkungen • keine Schutzmechanismen • keine klare Kontrolle über Output	• klarer Kontext • aktueller Stand beschrieben • Beispiele vorhanden • verständlicher Code gefordert	

Erkenntnis

Mehr Struktur = bessere Ergebnisse

Prompt (mit Struktur)

Kontext:

Wir bauen eine einfache stackbasierte Sprache in Java namens Jorth.Es gibt eine Methode run(String input), die ein Programm ausführt.Ein Stack speichert Werte.

Tokens sind durch Leerzeichen getrennt.

Aktueller Stand:

Die Methode run existiert, macht aber noch nichts und gibt aktuell "[]" zurück.Am Ende soll run den Stackinhalt als String zurückgeben, z. B. "[3, 4]".

Aufgabe:

Ergänze die Methode run so, dass Ganzzahlen erkannt und auf den Stack gelegt werden.

Beispiele:

Input: "3" → "[3]"

Input: "3 4" → "[3, 4]"

Wichtig:

- Keine komplette Umstrukturierung
- Keine neuen Klassen
- Keine externen Libraries
- Bestehende Struktur beibehalten
- Nur minimal notwendige Änderungen

Qualität:

- Einfacher, gut lesbarer Java-Code
- Keine unnötige Komplexität

- Ausgabe:
- Bitte liefere die vollständige Methode run.

Was ist noch zu tun?

- Prompt ausführen
- Erstellen des Testcodes
- Laufen lassen
- Einchecken nicht vergessen

- Diskussion:
  - Funktioniert es?
  - Ist der Code verständlich?
  - Halten Sie dies für gut?

Referenzcode

- Die Referenzimplementierung in Step_01 ignoriert Tokens, die keine Integer sind
- Das hatten wir so nicht spezifiziert.
- Damit ist es ein Beispiel, wo GenAI sich die Freiheit genommen hat, dies zu tun.
- GenAI hat mitgedacht.

Was könnte problematisch sein?

- keine Fehlerbehandlung
- nur Integer
- keine Trennung von Parsing und Logik
- Stack Implementiert

Ab jetzt

- Der existierende Code ist die Ausgangsbasis für den jeweils nächsten Schritt.
- Sie entwerfen die nächsten Prompts selbst, nehmen die Ergebnisse und testen das Ergebnis.
- Ich stelle am Ende jeden Schrittes die Musterlösung ins Repository
- Greifen Sie auf die Musterlösung zurück, falls ihr Code mal völlig "zerschossen" ist.

In a nutshell

- Wir erstellten den Prompt
- KI hat Code erzeugt
- Tests haben ihn verifiziert
- Git hat ihn gesichert

Prompt → Code → Test → Commit

Schritt 2 - Aufgabe

- Wir führen unseren ersten Command
- Ein + arbeitet auf dem Stack
- Kombiniert zwei Werte

Beispiele:	3 4 +	-> [7]
	10 2 +	-> [12]
	11 3 8 1 + + +	-> [23]

Schritt 2 – Testen miteinbeziehen

KI darf Tests hinzufügen,

aber nicht die bestehenden ändern.

Diskussion: Schritt 2

Warnung

- Erfolgreiche Tests können in die Irre leiten
- Bsp.: 3 5 + +
- Tests müssen verstanden werden
- Sie müssen sinnvolle Situationen testen
- Aber auch Grenzbereiche, Edge Cases, Fehlerfälle

Für Heute reicht unser Ansatz, aber für einen guten Interpreter wird dies benötigt.

Schritt 3

- Erweitern Sie das Programm um das – Command (Subtraktion)

Schritt 4

- Erweitern Sie das Programm um das . Command (Entspricht println)
- Das oberste Element das Stack wir auf die Console ausgegeben – aber der Stack bleibt unverändert

- Bsp.: "3 4 ." → Ausgabe: 4
  "3 4 + ." → Ausgabe: 7

Achtung

- Was fällt bei run-tests auf?

Weitere Technical Debts

- Da wir nur den Stackinhalt mit der run Methode zurückgeben, ist das Testen des . Commands nicht automatisiert möglich.

- Auch dies lassen wir im weiteren unberücksichtigt.

Schritt 5 - Strings

- Ein String ist alles zwischen zwei "

- Bsp.: "hello"
  "hello world"
  "123 456"

Schritt 5

- Bitte implementieren Sie die Erkennung und Speicherung von Strings

- Verarbeitung ist noch nicht möglich.

Schritt 6 – String Concatination

- Bitte implementieren Sie folgenden Funktionalität (String Verkettung mit + Command):

"ab" "cd" +	→ [abcd]
3 4 +	→ [7]
"ab" 3 +	→ vorerst unverändert

Schritt 6

- Ab jetzt kann das + Command

  Integer + Integer
  und
  String + String

Schritt 7 – bessere Ausgabe

- Ziel ist es, dass wir eine Ausgabe ohne Zeilenschaltung machen können. Wir benennen diese Command ._

- Bitte implementieren Sie das ._ Command, dass sich wie das . Command verhält, aber keine Zeilenschaltung vornimmt.

Schritt 7 – Was fällt auf?

- Stellen Sie durch das Testen die Probleme mit dieser Implementierung fest.

Probleme

- Top of Stack wird nicht entfernt
- Die Tests sind nicht unabhängig voneinander. Der Stack wird zwischen den Tests nicht geleert.

Schritt 8

- Implementieren Sie ein drop Command, dass den obersten Eintrag im Stack entfernt.
- Schreiben Sie die run Methode so um, dass sie nicht mehr static ist und passen Sie die Tests an.

Schritt 9 – dup Command

- Erstellen Sie ein dup Command, das den obersten Eintrag des Stack dupliziert

- Bsp.:

3 dup	→ [3, 3]
"ab" dup	→ [ab, ab]
dup	→ []

Schritt 10 - Comparison

- Implementieren Sie Vergleichsoperatoren (>, <, ==, !=). 0 für False, 1 für True

- Bsp.:

	3 4 <	→ [1]
	3 4 >	→ [0]
	3 3 ==	→ [1]
	3 4 !=	→ [1]

Step 11 – if else endif

- Erstellen Sie die Möglichkeit des Kontrollflusses durch das Implementieren von Abfragen via if else endif.
  (0 bedeutet False, alles anderen bedeutet True)

- Bsp: 1 if "yes" else "no" endif → [yes]

  0 if "yes" else "no" endif → [no]

Was überhaupt noch nicht bedacht wurde

- Java Version